format, the system programming language SPL, the high-level language EXPL, etc. The second phase involves Stages 13 to 19 and implements the scheduler of the operating system, hardware operations like disk interrupt handling and exception handling, and manages the console input. The third and final phase involves Stages 20 to 28 and implements system calls for process creation, termination and synchronisation, file creation, deletion, read and write, etc. Stage 28 of the third phase also enables eXpOS to support a two-core machine.

You need to install Flex (a lexical analyser generator), GNU Bison (a parser generator), *tar* (a utility for archive management) and *make* (a tool for build automation) before proceeding with the installation of eXpOS. You also need to install the package *libreadline-dev* before installing eXpOS. This library is available in Linux operating systems like Ubuntu, Debian, etc. This is where I had some difficulty with eXpOS. The package *readline-dev* is not available in Linux operating systems like Fedora. I have tried with equivalent packages like *readline-devel* on Fedora. But so far, I have been unable to install eXpOS on Fedora, and I will be very happy if somebody could help me do so. Stage 1 of the eXpOS roadmap provides the detailed steps to install the necessary tools to develop eXpOS. You can download the compressed installation file from the link provided in Stage 1, and unzip it with the utility *tar*. Figure 1 shows the terminal when the installation of the eXpOS packages have been completed successfully with the utility *make*.

After a successful execution, the directory named *myexpos* will contain the following sub-directories — *expl*, *spl*, *xfs-interface* and *xsm*. The directory *expl* has the files associated with the EXPL compiler. The directory *spl* has the files associated with the SPL compiler. The directory *xfs-interface* contains the files associated with the XFS interface. And the directory *xsm* contains the files associated with the eXperimental String Machine (XSM).

```
student@student: ~/myexpos
cc -g -c tokenize.c
cc -g -c disk.c
cc -g -c debug.c
cc -g -c exception.c
cc -g -o xsm lex.yy.o machine.o main.o simulator.o word.o memory.o registers.o t
okenize.o disk.o debug.o exception.o '-ll'
make[1]: Leaving directory '/home/student/myexpos/xsm'
 #
```

Figure 1: Successful installation of eXpOS packages

```
student@student: ~/myexpos/xsm
# ./xsm
ABCDEFGHIJKLM
Machine is halting.
#
```

Figure 2: Output of the program *msg.xsm*

It is impossible to summarise all the details regarding eXpOS in this short article. So let us try to understand how eXpOS works by running three programs written in the following three programming languages — XSM assembly language, SPL system programming language and EXPL high-level programming language. Please note that in order to run these programs, we have to develop different modules of the operating system eXpOS, itself. But we will only explore the three programs and their execution to understand the power of eXpOS.

First, let us consider the XSM assembly language program called *msg.xsm* shown below. The program tries to print the string 'ABCDEFGHIJKLMNOP'.

```
MOV R0, "ABCDEFGHIJKLMNOP"
MOV R16, R0
PORT P1, R16
OUT
HALT
```

Notice that the XSM assembly language is similar to the Intel syntax of the x86 assembly language. This program can be executed after developing eXpOS up to Stage 3. Notice that in this stage, the program can only be executed as the OS start-up code. We can create the program file *msg.xsm* using some text editor, and then save it to the computer in which we are simulating the eXperimental String Machine (XSM). A string based simulated file system called Experimental File System (eXpFS) is used by eXpOS to save files. The XFS interface is used to transfer files from the real file system to the simulated file system, eXpFS. After loading the program *msg.xsm* to eXpFS as OS start-up code, we can run the XSM machine with the command *xsm*. Figure 2 shows the output of the program *msg.xsm*. In this figure, we can see that the string printed is 'ABCDEFGHIJKLM' and not 'ABCDEFGHIJKLMNOP'. This gives us a hint about the capacity of the string-based storage of the eXpFS file system.

Now, let us consider the SPL program called *count.spl* shown below. The program *count.spl*, which prints the numbers from 1 to 10, can be executed after developing the eXpOS up to Stage 4.

```
alias count R0;
count = 1;
while(count <= 10) do
  print count;
  count = count + 1;
endwhile;
```

Notice that the XSM machine cannot execute SPL programs directly; so we will use the SPL compiler to compile the program *count.spl* to obtain the XSM assembly language program *count.xsm*. This assembly program is also loaded as OS start-up code to the eXpFS file system by using the XFS interface for execution. Figure 3 shows the output